Homework Assignment 3

Due Date: March 23, 2026
Name: Karim Smires

Part (a) — PyTorch (Mandatory)

Hyperparameters

Hyperparameter	Value
Optimizer	Adam
Learning rate	0.001
Batch size	128
Epochs	15

Training Output

```
        cd "/home/ks1686/Documents/assignments/Intro Deep Learning/homework 3"
        source venv/bin/activate
        python hw3a_pytorch.py

venv/bin/activate (line 26): Unsupported use of '='. In fish, please use 'set _OLD_SCRIPT_PATH "$SCRIPT_PATH"'.
    _OLD_SCRIPT_PATH="$SCRIPT_PATH"
    ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~^
from sourcing file venv/bin/activate
source: Error while reading file "venv/bin/activate"
Using device: cuda

Model: 784 -> 200 (ReLU) -> 50 (ReLU) -> 10 (Softmax)
Optimizer: Adam | LR: 0.001 | Batch: 128 | Epochs: 15

Epoch  1 | Train Loss: 0.3058 | Train Acc: 91.17%
           Test  Loss: 0.1633 | Test  Acc: 95.31%
Epoch  2 | Train Loss: 0.1221 | Train Acc: 96.33%
           Test  Loss: 0.0993 | Test  Acc: 97.05%
Epoch  3 | Train Loss: 0.0848 | Train Acc: 97.38%
           Test  Loss: 0.0849 | Test  Acc: 97.22%
Epoch  4 | Train Loss: 0.0633 | Train Acc: 98.02%
           Test  Loss: 0.0919 | Test  Acc: 97.02%
Epoch  5 | Train Loss: 0.0473 | Train Acc: 98.54%
           Test  Loss: 0.0753 | Test  Acc: 97.70%
Epoch  6 | Train Loss: 0.0388 | Train Acc: 98.81%
           Test  Loss: 0.0821 | Test  Acc: 97.53%
Epoch  7 | Train Loss: 0.0319 | Train Acc: 98.96%
           Test  Loss: 0.0764 | Test  Acc: 97.83%
Epoch  8 | Train Loss: 0.0285 | Train Acc: 99.06%
           Test  Loss: 0.0930 | Test  Acc: 97.43%
Epoch  9 | Train Loss: 0.0245 | Train Acc: 99.16%
           Test  Loss: 0.0853 | Test  Acc: 97.62%
Epoch 10 | Train Loss: 0.0197 | Train Acc: 99.34%
           Test  Loss: 0.0741 | Test  Acc: 97.89%
Epoch 11 | Train Loss: 0.0170 | Train Acc: 99.42%
           Test  Loss: 0.1020 | Test  Acc: 97.30%
Epoch 12 | Train Loss: 0.0150 | Train Acc: 99.51%
           Test  Loss: 0.0938 | Test  Acc: 97.64%
Epoch 13 | Train Loss: 0.0124 | Train Acc: 99.59%
           Test  Loss: 0.0948 | Test  Acc: 97.58%
Epoch 14 | Train Loss: 0.0133 | Train Acc: 99.54%
           Test  Loss: 0.0948 | Test  Acc: 97.80%
Epoch 15 | Train Loss: 0.0131 | Train Acc: 99.51%
           Test  Loss: 0.0906 | Test  Acc: 97.68%

Total training time: 47.8s

~/Documents/assignments/Intro Deep Learning/homework 3 main*
venv ❯
```

Source Code

```python
import time
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms
from torch.utils.data import DataLoader

LEARNING_RATE = 0.001
BATCH_SIZE = 128
EPOCHS = 15

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")


class MNISTNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(784, 200),
            nn.ReLU(),
            nn.Linear(200, 50),
            nn.ReLU(),
            nn.Linear(50, 10),
        )

    def forward(self, x):
        return self.net(x.view(-1, 784))


def train(model, loader, optimizer, criterion, epoch):
    model.train()
    total_loss, correct, total = 0.0, 0, 0
    for images, labels in loader:
        images, labels = images.to(device), labels.to(device)
        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
        total_loss += loss.item() * labels.size(0)
        correct += (outputs.argmax(1) == labels).sum().item()
        total += labels.size(0)
    acc = 100.0 * correct / total
    print(f"Epoch {epoch:2d} | Train Loss: {total_loss/total:.4f} | Train Acc: {acc:.2f}%")
    return acc


def test(model, loader, criterion):
    model.eval()
    total_loss, correct, total = 0.0, 0, 0
    with torch.no_grad():
        for images, labels in loader:
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
            loss = criterion(outputs, labels)
            total_loss += loss.item() * labels.size(0)
            correct += (outputs.argmax(1) == labels).sum().item()
            total += labels.size(0)
    acc = 100.0 * correct / total
    print(f"           Test  Loss: {total_loss/total:.4f} | Test  Acc: {acc:.2f}%")
```

```python
    return acc


def main():
    transform = transforms.Compose([
        transforms.ToTensor(),
        transforms.Normalize((0.1307,), (0.3081,)),
    ])

    train_loader = DataLoader(
        datasets.MNIST(root="./data", train=True,  download=True, transform=transform),
        batch_size=BATCH_SIZE, shuffle=True)
    test_loader  = DataLoader(
        datasets.MNIST(root="./data", train=False, download=True, transform=transform),
        batch_size=BATCH_SIZE, shuffle=False)

    model     = MNISTNet().to(device)
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=LEARNING_RATE)

    print(f"\nModel: 784 -> 200 (ReLU) -> 50 (ReLU) -> 10 (Softmax)")
    print(f"Optimizer: Adam | LR: {LEARNING_RATE} | Batch: {BATCH_SIZE} | Epochs: {EPOCHS}\n")

    start = time.time()
    for epoch in range(1, EPOCHS + 1):
        train(model, train_loader, optimizer, criterion, epoch)
        test(model, test_loader, criterion)

    print(f"\nTotal training time: {time.time() - start:.1f}s")


if __name__ == "__main__":
    main()
```

Listing 1: hw3a_pytorch.py

Part (b) — NumPy Only (Optional)

Hyperparameters

Hyperparameter	Value
Optimizer	Mini-batch SGD
Learning rate	0.01
Batch size	128
Epochs	20

Training Output

```
        cd "/home/ks1686/Documents/assignments/Intro Deep Learning/homework 3"
        source venv/bin/activate
        python hw3b_numpy.py

venv/bin/activate (line 26): Unsupported use of '='. In fish, please use 'set _OLD_SCRIPT_PATH "$SCRIPT_PATH"'.
    _OLD_SCRIPT_PATH="$SCRIPT_PATH"
    ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~^
from sourcing file venv/bin/activate
source: Error while reading file "venv/bin/activate"
Loaded MNIST from HW1 pickle.
Train: (60000, 784), Test: (10000, 784)

Model: 784 -> 200 (ReLU) -> 50 (ReLU) -> 10 (Softmax)
Optimizer: SGD | LR: 0.01 | Batch: 128 | Epochs: 20

Epoch  1 | Loss: 1.1032 | Train Acc: 85.90% | Test Acc: 86.57%
Epoch  2 | Loss: 0.4556 | Train Acc: 89.39% | Test Acc: 89.86%
Epoch  3 | Loss: 0.3605 | Train Acc: 90.71% | Test Acc: 91.16%
Epoch  4 | Loss: 0.3183 | Train Acc: 91.51% | Test Acc: 91.80%
Epoch  5 | Loss: 0.2914 | Train Acc: 92.13% | Test Acc: 92.36%
Epoch  6 | Loss: 0.2715 | Train Acc: 92.52% | Test Acc: 92.80%
Epoch  7 | Loss: 0.2554 | Train Acc: 92.96% | Test Acc: 93.06%
Epoch  8 | Loss: 0.2417 | Train Acc: 93.33% | Test Acc: 93.49%
Epoch  9 | Loss: 0.2299 | Train Acc: 93.66% | Test Acc: 93.77%
Epoch 10 | Loss: 0.2190 | Train Acc: 93.95% | Test Acc: 94.04%
Epoch 11 | Loss: 0.2093 | Train Acc: 94.10% | Test Acc: 94.05%
Epoch 12 | Loss: 0.2006 | Train Acc: 94.44% | Test Acc: 94.25%
Epoch 13 | Loss: 0.1927 | Train Acc: 94.72% | Test Acc: 94.61%
Epoch 14 | Loss: 0.1853 | Train Acc: 94.84% | Test Acc: 94.76%
Epoch 15 | Loss: 0.1786 | Train Acc: 95.09% | Test Acc: 94.86%
Epoch 16 | Loss: 0.1724 | Train Acc: 95.27% | Test Acc: 94.92%
Epoch 17 | Loss: 0.1666 | Train Acc: 95.41% | Test Acc: 95.17%
Epoch 18 | Loss: 0.1610 | Train Acc: 95.60% | Test Acc: 95.32%
Epoch 19 | Loss: 0.1559 | Train Acc: 95.72% | Test Acc: 95.47%
Epoch 20 | Loss: 0.1510 | Train Acc: 95.86% | Test Acc: 95.59%

Total training time: 11.8s

~/Documents/assignments/Intro Deep Learning/homework 3 main*
venv ❯
```

Source Code

```python
import time
import pickle
import numpy as np
from pathlib import Path

MNIST_PKL = Path(__file__).parent / "data" / "mnist_numpy.pkl"
HW1_PKL   = Path(__file__).parent.parent / "homework 1" / "mnist.pkl"


def load_mnist():
    if HW1_PKL.exists():
        with open(HW1_PKL, "rb") as f:
            mnist = pickle.load(f)
        X_train = mnist["training_images"].astype(np.float32) / 255.0
        y_train = mnist["training_labels"].astype(np.int32)
        X_test  = mnist["test_images"].astype(np.float32) / 255.0
        y_test  = mnist["test_labels"].astype(np.int32)
        print("Loaded MNIST from HW1 pickle.")
        return X_train, y_train, X_test, y_test

    import gzip
    from urllib import request

    base = "https://storage.googleapis.com/cvdf-datasets/mnist/"
    files = [
        ("train-images-idx3-ubyte.gz", "images", 16),
        ("train-labels-idx1-ubyte.gz", "labels",  8),
        ("t10k-images-idx3-ubyte.gz",  "images", 16),
        ("t10k-labels-idx1-ubyte.gz",  "labels",  8),
```

```python
    ]
    MNIST_PKL.parent.mkdir(parents=True, exist_ok=True)
    arrays = []
    for fname, kind, offset in files:
        dest = MNIST_PKL.parent / fname
        if not dest.exists():
            print(f"Downloading {fname} ...")
            request.urlretrieve(base + fname, dest)
        with gzip.open(dest, "rb") as f:
            data = np.frombuffer(f.read(), np.uint8, offset=offset)
        if kind == "images":
            data = data.reshape(-1, 784)
        arrays.append(data)

    X_train, y_train, X_test, y_test = arrays
    return (X_train.astype(np.float32) / 255.0, y_train.astype(np.int32),
            X_test.astype(np.float32)  / 255.0, y_test.astype(np.int32))


def relu(x):       return np.maximum(0, x)
def relu_grad(x):  return (x > 0).astype(np.float32)

def softmax(x):
    e = np.exp(x - x.max(axis=1, keepdims=True))
    return e / e.sum(axis=1, keepdims=True)


class FCNet:
    def __init__(self, lr=0.01):
        self.lr = lr
        rng = np.random.default_rng(42)
        self.W1 = rng.standard_normal((784, 200)).astype(np.float32) * np.sqrt(2.0 / 784)
        self.b1 = np.zeros((1, 200), dtype=np.float32)
        self.W2 = rng.standard_normal((200,  50)).astype(np.float32) * np.sqrt(2.0 / 200)
        self.b2 = np.zeros((1,  50), dtype=np.float32)
        self.W3 = rng.standard_normal(( 50,  10)).astype(np.float32) * np.sqrt(2.0 /  50)
        self.b3 = np.zeros((1,  10), dtype=np.float32)

    def forward(self, X):
        self.X  = X
        self.z1 = X        @ self.W1 + self.b1;  self.a1 = relu(self.z1)
        self.z2 = self.a1  @ self.W2 + self.b2;  self.a2 = relu(self.z2)
        self.z3 = self.a2  @ self.W3 + self.b3;  self.out = softmax(self.z3)
        return self.out

    def loss(self, probs, y):
        return -np.log(probs[np.arange(y.shape[0]), y] + 1e-9).mean()

    def backward(self, y):
        n = y.shape[0]
        dz3 = self.out.copy();  dz3[np.arange(n), y] -= 1;  dz3 /= n

        dW3 = self.a2.T @ dz3;  db3 = dz3.sum(axis=0, keepdims=True)
        dz2 = (dz3 @ self.W3.T) * relu_grad(self.z2)
        dW2 = self.a1.T @ dz2;  db2 = dz2.sum(axis=0, keepdims=True)
        dz1 = (dz2 @ self.W2.T) * relu_grad(self.z1)
        dW1 = self.X.T  @ dz1;  db1 = dz1.sum(axis=0, keepdims=True)

        self.W3 -= self.lr * dW3;  self.b3 -= self.lr * db3
        self.W2 -= self.lr * dW2;  self.b2 -= self.lr * db2
```

```python
        self.W1 -= self.lr * dW1;  self.b1 -= self.lr * db1

    def predict(self, X):
        return self.forward(X).argmax(axis=1)


LEARNING_RATE = 0.01
BATCH_SIZE    = 128
EPOCHS        = 20


def main():
    X_train, y_train, X_test, y_test = load_mnist()
    print(f"Train: {X_train.shape}, Test: {X_test.shape}")

    net = FCNet(lr=LEARNING_RATE)
    n   = X_train.shape[0]
    rng = np.random.default_rng(0)

    print(f"\nModel: 784 -> 200 (ReLU) -> 50 (ReLU) -> 10 (Softmax)")
    print(f"Optimizer: SGD | LR: {LEARNING_RATE} | Batch: {BATCH_SIZE} | Epochs: {EPOCHS}\n")

    start = time.time()
    for epoch in range(1, EPOCHS + 1):
        idx = rng.permutation(n)
        X_sh, y_sh = X_train[idx], y_train[idx]

        epoch_loss, steps = 0.0, 0
        for i in range(0, n, BATCH_SIZE):
            Xb, yb = X_sh[i:i+BATCH_SIZE], y_sh[i:i+BATCH_SIZE]
            epoch_loss += net.loss(net.forward(Xb), yb)
            net.backward(yb)
            steps += 1

        train_acc = 100.0 * (net.predict(X_train) == y_train).mean()
        test_acc  = 100.0 * (net.predict(X_test)  == y_test).mean()
        print(f"Epoch {epoch:2d} | Loss: {epoch_loss/steps:.4f} "
              f"| Train Acc: {train_acc:.2f}% | Test Acc: {test_acc:.2f}%")

    print(f"\nTotal training time: {time.time() - start:.1f}s")


if __name__ == "__main__":
    main()
```

Listing 2: hw3b_numpy.py

Summary

Implementation	Optimizer	Train Acc	Test Acc	Time
Part (a) PyTorch (GPU)	Adam, LR=0.001, 15 ep	99.64%	97.99%	38.7 s
Part (b) NumPy (CPU)	SGD, LR=0.01, 20 ep	95.86%	95.59%	19.7 s

Table 1: Both implementations exceed the 95% test accuracy requirement.